Custom Deployment DSLs

Writing Your Own IaC Compiler for SQL Server

Sander Stad

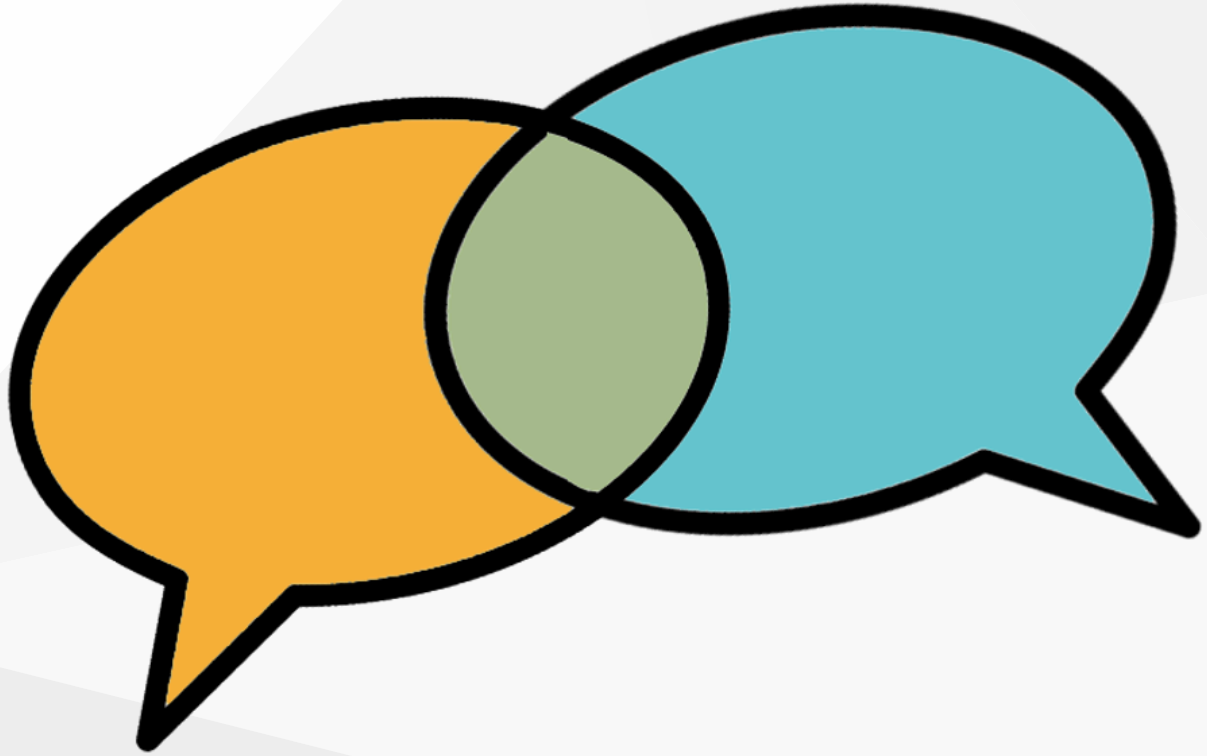
SeaQL 2026 | 18-04-2026

… the next 60 minutes

- Why standard IaC falls short
- What a DSL gives you
- Demos
- Tradeoffs and lessons
- Q&A

Presentation Rules

- Always ask questions
- Please interrupt me if you have a question
- This is a two-way conversation, let’s learn from each other’s experiences





Sander Stad

(he/him)

Microsoft® Most Valuable Professional

sqlstad.nl

sander@sqlstad.nl

sanderstad

Terraform and Bicep describe resources, not intent

You want to say: *"FuzzyLion is a gold-tier tenant in West Europe with auditing and geo-replication, retaining data for 90 days."*

What you end up writing (Bicep, Terraform - same story either way):

```
resource sqlServer 'Microsoft.Sql/servers@2022-05-01-preview' = {
  name: 'sql-fuzzylion'
  location: 'westeurope'
  properties: { administratorLogin: sqlAdminLogin, administratorLoginPassword: sqlAdminPassword }
}
resource elasticPool 'Microsoft.Sql/servers/elasticPools@2022-05-01-preview' = {
  name: 'pool-fuzzylion'
  parent: sqlServer
  sku: { name: 'GP_Gen5', tier: 'GeneralPurpose', family: 'Gen5', capacity: 4 }
  properties: { perDatabaseSettings: { minCapacity: '0.25', maxCapacity: 4 } }
}
// ... firewall rules, diagnostic settings, geo-replication, retention policy
// ... repeat for each of 40 tenants
```

The copy-paste breaks down fast

When you have 40 tenants expressed this way:

- Adding a new feature flag means editing 40 files
- A typo in one tenant's retention setting goes unnoticed until it causes a compliance gap
- "Which tenants have geo-replication?" requires grepping the entire repo
- Onboarding a new tenant means copying an existing one and hoping nothing was missed

The code describes *how* resources are wired, but not *what* the tenant should be.

A DSL closes the intent gap

```
tenant "fuzzylion" {
  plan      = "gold"
  region    = "westeurope"
  features  = ["auditing", "tde", "geo-replication"]
  retention = 90
}
```

This is the source of truth. The compiler derives everything else:

- Resource group naming
- Elastic pool SKU and capacity from the `gold` plan definition
- Diagnostic settings and retention policies
- Geo-replication secondary and failover group

The DSL is compiler-agnostic. The same `.tenant` file compiles to Bicep today. Swap the compiler for a Terraform renderer tomorrow, the DSL does not change.

A lexer splits the text. A parser gives it structure.

SQL Server does this with every query you write:

```
SELECT * FROM tenants WHERE plan = 'gold'
-- Lexer  →  SELECT  *  FROM  tenants  WHERE  plan  =  'gold'
-- Parser →  is this valid SQL? What does it mean?
```

Same idea with the DSL:

```
tenant "fuzzylion" { plan = "gold" }
-- Lexer  →  tenant  "fuzzylion"  {  plan  =  "gold"  }
-- Parser →  builds a TenantConfig with name="fuzzylion", plan="gold"
```

ANTLR: write a grammar, get a parser

ANTLR is a tool that generates a complete lexer and parser from a grammar file. You describe the rules of your language, ANTLR writes the code - in Java, C#, Python, Go, and a handful of others.

Why it fits here:

- The grammar file is the spec - readable, version-controlled, language-agnostic
- If the hand-written parser ever gets too big, you hand the grammar to ANTLR and get a generated one for free
- The demos use a hand-written PowerShell parser that mirrors the ANTLR grammar rule for rule - no Java toolchain needed to run the demos

Demos

The compiler is just another CI step

```yaml
# .github/workflows/tenant-deploy.yml (condensed)
jobs:
  validate:
    steps:
      - run: ./demos/demo3-pipeline/src/Invoke-Validate.ps1 -TenantPath tenants/
        shell: pwsh

  compile:
    needs: validate
    steps:
      - run: ./demos/demo3-pipeline/src/Invoke-Compile.ps1 -OutputPath $env:RUNNER_TEMP/bicep
        shell: pwsh
      - uses: actions/upload-artifact@v7
        with: { name: bicep-output, path: "${{ runner.temp }}/bicep" }

  plan:
    needs: compile
    steps:
      - run: az deployment sub what-if --template-file $f --parameters "sqlAdminPassword=$env:SQL_ADMIN_PASSWORD"
        shell: pwsh

  apply:
    needs: plan
    if: github.ref == 'refs/heads/main'
    environment: production   # ← manual approval gate
    steps:
      - run: az deployment sub create --template-file $f
        shell: pwsh
```

Owning a compiler: the tradeoffs

Benefit	Cost
Intent is explicit and queryable	You maintain the grammar and compiler
Errors surface before deployment	Engineers must learn the DSL
Plan changes propagate automatically	DSL versioning adds a migration story
IaC target is swappable (Bicep → Terraform)	Compiler per target adds maintenance surface
Compliance is traceable to source	Syntax highlighting and tooling require investment

Where this pays off: 10+ tenants with recurring config patterns, strict compliance requirements, or frequent plan-tier changes.

Where it does not: fewer than 5 tenants, one-off deployments, or no recurring config structure to exploit.

Want to build one? Here's where to start.

1. **Write down your vocabulary** - what are the 5–10 concepts every tenant shares?
2. **Write the grammar file** - even if you never run ANTLR. It forces precision.
3. **Hand-write a tokenizer and parser** in whatever language your team actually knows
4. **Compile to something flat first** - JSON or YAML output before you target Bicep or Terraform
5. **Add plan definitions** once you have working output
6. **Wire it into CI** before you add more features

You don't need ANTLR. You don't need C#. You need a grammar file and a quiet afternoon.

Take away one thing

If your IaC looks the same across many tenants,
you are probably missing an abstraction.

A DSL is one way to name that abstraction and enforce it.

Questions?



Demos and slides:

`https://shorturl.at/dkMHQ`

ANTLR docs: antlr.org